A Real-Time FPGA Implementation of the LCMV Algorithm for Target Classification in Hyperspectral Images Using LDL Decomposition

Pedro Palacios, Daniel Báscones, Carlos González, and Daniel Mozos, *Member, IEEE*

***Abstract*— Since the advent of air and space-borne flight, remotely sensed images of the Earth's surface have changed how the world is perceived: meteorology, navigation, surveillance, and environmental sciences are some of the areas, in which technology has fundamentally changed. Hyperspectral images extend the possibilities even further by capturing wavelengths outside the visible spectrum. This is particularly valuable for target detection, often performed using drones, satellites, or other unmanned aerial vehicles (UAVs). In many applications such as surveillance and security, it is critical to perform this detection in real-time. Response times can be optimized by performing these calculations on-board, which introduces additional power, radiation tolerance, and resource constraints. To that end, embedded devices such as field-programmable gate arrays (FPGAs) are ideal candidates, as their reprogrammable logic allows for a great degree of acceleration while having low power consumption. In this article, a real-time FPGA implementation of the linearly constrained minimum variance (LCMV) algorithm for target classification is presented. First, an analysis is performed to explore the most parallelizable and accurate numerical methods that fit within the resource constraints of FPGAs, and finally selecting LDL decomposition. Then, hardware modules for LCMV classification are designed in SystemVerilog, focusing on incremental correlation matrix computation and a solver for LDL matrix decomposition. The proposed implementation on a radiation-tolerant Xilinx XQRKU060 is compared to an embedded CUDA-accelerated graphics processing unit (GPU) and an embedded central processing unit (CPU), showing turnaround time improvements of 7.25× and 825×, respectively. It runs 15× faster than real-time, with sufficient margin to accommodate the next generation of hyperspectral sensors.**

***Index Terms*— Field-programmable gate array (FPGA), hyperspectral, LDL decomposition, linearly constrained minimum variance (LCMV), real-time, target classification.**

I. Introduction

REMOTELY sensed hyperspectral images [1], [2] are an excellent tool that enables a variety of analyses that would otherwise be too costly, or outright impossible to perform on the ground. A portion of the electromagnetic spectrum, usually starting below and ending above the visible wavelength, is split into equally spaced bands. Each pixel of a hyperspectral image contains intensity information of each band, introducing a spectral dimension to imaging technologies.

Two of the most widespread applications are target detection [3] and target classification [4], for which many techniques have been developed over the years. Hyperspectral target detection [5] involves recognizing the presence of objects, with a focus on identifying if a single type of object is present or not, whereas classification [6] categorizes pixels in a hyperspectral image into multiple predefined classes.

From traditional classification models to neural network-based architectures, using hyperspectral images allows for a greater degree of detail when identifying different materials, elements, or objects within a scene. Hyperspectral target detection and classification algorithms [7], [8], [9], [10], [11], [12] have found widespread applications in different areas: medical diagnosis [13], mineral detection [14], water quality monitoring [15], and crop yield optimization [16] are only some examples of the current applications. These analyses, generally, require extensive computation due to the inherent size of hyperspectral data.

When used for remote sensing, hyperspectral images are captured from aerial or space platforms such as unmanned aerial vehicles (UAVs), drones, planes, or satellites [17], [18]. These platforms carry the hyperspectral sensor, position themselves in sight of the region of interest, and capture the image, which can then be stored or sent back to ground stations for processing. However, in critical scenarios, such as wildfire prevention [19], military, security, or emergency applications [20], [21], this pipeline might not be adequate since it introduces delays, which might not be acceptable if a real-time solution is required. In such cases, on-board processing becomes a necessity. If the processing speed is slower than the capture rate, not all frames can be processed, leading to either information loss or an infinite increase in response time.

Graphics processing units (GPUs) and field-programmable gate arrays (FPGAs) have generally been the platforms of choice [22], [23] when real-time capabilities are needed. While the former offer fast development cycles and throughput, the latter offer great customizability, performance, and most importantly, have low power requirements that are key in

Manuscript received 15 March 2024; revised 14 June 2024; accepted 19 July 2024. Date of publication 29 July 2024; date of current version 5 August 2024. This work was supported in part by Ministerio de Economía, Industria y Competitividad (MINECO) under Grant PID2020-112916GB-I00 and in part by La Caixa Foundation (ID 100010434) under Grant LCF/BQ/EU23/12010073. *(Corresponding author: Pedro Palacios.)*

The authors are with the Department of Computer Architecture and Automatics (DACYA), Complutense University of Madrid, 28035 Madrid, Spain (e-mail: pedpalac@ucm.es; danibasc@ucm.es; carlosgo@ucm.es; mozos@ucm.es).

Digital Object Identifier 10.1109/TGRS.2024.3434423

1558-0644 © 2024 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 05:56:59 UTC from IEEE Xplore. Restrictions apply.

enabling on-orbit computation, a keystone in hyperspectral processing [24]. Classification [25], anomaly [26], and target detection [27] are some of the current state-of-the-art applications that benefit from FPGA acceleration. Furthermore, commercial radiation-tolerant devices exist [28], which simplify the integration of such processing in vulnerable platforms like satellites, as they do not require additional radiation protection.

In this work, our aim is to provide a solution for target classification in real-time on an embedded device. Timing, power, and radiation constraints are considered, so that it can be integrated in the same platform as the sensor, providing valuable data with an almost instant turnaround of results. The linearly constrained minimum variance (LCMV) [10] classification algorithm using LDL decomposition [29] for target classification in hyperspectral images is selected, and implemented on a radiation-tolerant XQRKU060 FPGA.

First, the theory behind the algorithm is studied, in order to derive a version which, while maintaining the results, is more apt for FPGA implementation. In this aspect, several numerical methods are taken into consideration, and evaluated based on their resource utilization, performance, and accuracy, finally selecting LDL decomposition.

Then, special care is taken to split the algorithm into reusable modules, which are implemented and optimized for an FPGA, using a low-level RTL design for maximum control of the generated logic.

Finally, the hardware is benchmarked against commercial-off-the-shelf (COTS) central processing unit (CPU), embedded CPU, and embedded GPU implementations, showing that indeed FPGA real-time performance is possible and that FPGAs outperform all platforms in turnaround times for this specific algorithm, making them a great option for hyperspectral target classification. The results are also contextualized to analyze which platform might be better suited for different requirements.

A. State of the Art

In the context of target detection and classification, constrained energy minimization (CEM) and LCMV are two of the most popular approaches. Studies about real-time hardware implementation of CEM target detection have been presented in [30], [31], and [32]. Several authors have proposed using QR decomposition for a real-time hardware implementation of LCMV classification [10], [33]. Both approaches, however, have some drawbacks when considering computation of hyperspectral images.

The CEM real-time implementation involves incrementally updating the correlation matrix inverse R^{-1} by using the Sherman–Morrison and Woodbury formulas [34], which require each incremental modification of R to be of the type $R' = R + uv^{\mathsf{T}}$. This assumption does not generalize from CEM detection to LCMV classification, as, aside from computing the matrix R^{-1}, the inverse $(T^{\mathsf{T}}R^{-1}T)^{-1}$ must also be computed.

Furthermore, no general results about the accuracy and stability of the Sherman–Morrison formula have been published to the best of our knowledge. However, some results have been published [35] showing that LDL decomposition achieves better accuracy compared to other matrix-inversion-based methods. The Sherman–Morrison formula is a matrix-inversion method, as it first computes the inverse of the correlation matrix R^{-1} and then multiplies it by the coefficient matrix, instead of directly solving the linear system.

As for the LCMV implementations, those proposals involve either the costly computation of square roots or systolic arrays. The proposed systolic arrays for QR decomposition [36] are too resource-intensive for modern FPGAs given the large number of hyperspectral channels in current hyperspectral sensors. Indeed, in [36], the QR decomposition of matrices up to 10×10 is implemented on FPGAs, while the Hydice sensor [37] captures 169 spectral bands, and the AVIRIS sensor [38] produces images containing 224 spectral bands.

We explore two other linear solver methods: Gauss–Jordan and LDL decomposition. We perform a fixed-point precision analysis to compare how these two numerical methods and Sherman–Morrison matrix inversion affect the accuracy of the full LCMV classifier system, and we implement in hardware the most accurate one, LDL decomposition.

Although a systolic array implementation of LDL decomposition [39] has also been proposed, only matrices of size up to 8×8 are tested in the FPGA implementation, and the systolic array approach does not scale to 224×224 correlation matrices. Our proposed hardware LDL implementation is not based on systolic arrays, and the full LCMV classifier fits in commercial radiation-tolerant FPGAs such as Xilinx XQRKU060, capable of achieving real-time processing.

B. Main Contributions and Organization of the Article

The main contributions of this article include the following.

1) An exploration of different numerical methods for solving the linear systems required for LCMV classification, including the reduction of the LCMV classification problem to LDL decomposition.
2) A fixed-point precision analysis comparing how Gauss–Jordan matrix inversion, Sherman–Morrison matrix inversion, and LDL decomposition affect the final LCMV classification results in real hyperspectral images.
3) A custom hardware implementation of the LDL decomposition and LCMV classification algorithms adequate for the matrix sizes required by current hyperspectral sensors such as Hydice and AVIRIS is described, with optimized matrix storage structures and shared hardware arithmetic units.
4) A comparison of the FPGA implementation against optimized GPU and CPU implementations targeting embedded devices.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 05:56:59 UTC from IEEE Xplore. Restrictions apply.

The proposed techniques, combining both algorithmic and numerical optimizations, achieve classification accuracy on par with the reference software model. Furthermore, acceleration factors of over 7.25× and 825× are achieved when compared to the most optimized embedded GPU and embedded CPU code, respectively. This is the result of combining the on-the-fly computation of the correlation matrix, along with the hardware acceleration of the LDL and matrix multiplication steps.

The remainder of this article is organized as follows. In Section II, we examine the LCMV algorithm in detail and explore different numerical method alternatives. Section III focuses on the details of our proposed hardware implementation using LDL decomposition. Section IV presents the experimental results derived from our hardware implementation, and compares it to several CPU and GPU implementations. Section V offers a conclusion summarizing the key findings.

II. LCMV Algorithm

A. LCMV Classifier

A hyperspectral image is represented by a set of samples $\{\vec{r}_1, \ldots, \vec{r}_N\}$, where each sample is an L-dimensional vector, being L the number of hyperspectral bands of each pixel, that is, $\vec{r}_i \in \mathbb{R}^L$. Let K be the number of target signatures, the target matrix $T \in \mathcal{M}_{L,K}(\mathbb{R})$ is defined as the matrix formed by the column vectors $\vec{t}_i \in \mathbb{R}^L$ of the spectral signature of each target.

The LCMV target classifier [10] outputs an M-dimensional vector, where M is the number of classification classes. Note that M can be different from K. If $M < K$, then several target signatures of interest will correspond to the same classification class. The LCMV classifier can also be used for target visualization: if $M = 3$, an RGB color can be assigned to each target and the output of the classifier will be a visualization image that can be used for target discrimination.

The LCMV classifier is linear, which means that given a hyperspectral signature $\vec{x}$, the output of the classifier will be $\vec{y}^{\mathsf{T}} = \vec{x}^{\mathsf{T}} W$ where $W \in \mathcal{M}_{L,M}(\mathbb{R})$ is the weight matrix of the linear classifier.

Each target $\vec{t}_i$ will have a pre-defined associated classification output vector $\vec{c}_i \in \mathbb{R}^M$. All these vectors can be grouped into a matrix $C \in \mathcal{M}_{K,M}(\mathbb{R})$ where the ith column of C is $\vec{c}_i$. With this compact representation, the relation between the desired targets and their classification output can be expressed as $T^{\mathsf{T}} W = C$.

Once the outputs of the pre-defined targets are fixed, the LCMV algorithm finds a weight matrix W such that the average energy of the filter is minimized. In [40], it is proven that the optimal weight is W^* as shown as follows, where $R = (1/N)\sum_{j=1}^{N} \vec{r}_j \vec{r}_j^{\mathsf{T}}$ is the correlation matrix of the samples:

$$W^* = R^{-1} T \left(T^{\mathsf{T}} R^{-1} T\right)^{-1} C. \tag{1}$$

Note the difference between LCMV and CEM: if we are only interested in detecting a single target (CEM), then the matrix T becomes a vector $\vec{t}$, and the optimal solution vector $\vec{w}$ is given by

$$w^* = \frac{R^{-1}\vec{t}}{\vec{t}^{\,\mathsf{T}} R^{-1} \vec{t}}. \tag{2}$$

In the CEM case, R^{-1} is the only inverse matrix needed, which can be computed incrementally using the Sherman–Morrison and Woodbury formulas [34] as shown in [30], [31], and [32]. However, in the LCMV case, $(T^{\mathsf{T}} R^{-1} T)^{-1}$ must also be computed, which cannot be easily computed incrementally. Therefore, to use the same matrix inversion technique for both cases in order to ease implementation, share resources, and reduce FPGA utilization, we also explore other numerical methods.

B. Numerical Methods for Linear System Solving

Equation (1) uses matrix inversion. Looking more closely at the equation, there is no need to invert matrices, linear systems can be solved instead. Denoting $\texttt{lsolve}(A, B) = X$, where $AX = B$, then $R^{-1}T = \texttt{lsolve}(R, T)$. Furthermore, $(T^{\mathsf{T}} R^{-1} T)^{-1} C = \texttt{lsolve}(T^{\mathsf{T}} \texttt{lsolve}(R, T), C)$.

There are several approaches to solving the two linear systems that appear in (1).

1) Gauss–Jordan: By performing elementary row operations on a matrix, the Gauss–Jordan elimination algorithm [29] is able to directly compute the inverse of a given matrix, which can be later multiplied by another matrix or vector to solve a linear system.

First, the input matrix is horizontally augmented with an identity matrix, and then elementary row operations are performed on the augmented matrix until the left side of the augmented matrix is converted into the identity matrix. The output inverted matrix is found on the right side of the augmented matrix.

This method is highly parallelizable, and both pseudocode and hardware implementations can be found in [41]. However, as it is exposed in Section II-C, Gauss–Jordan matrix inversion has in many cases worse accuracy than other numerical methods.

2) QR Factorization: Every real square matrix A can be decomposed [42] as the product of an orthogonal matrix Q (which means that $Q^{\mathsf{T}} = Q^{-1}$) and an upper triangular matrix R, resulting in $A = QR$. After this decomposition, solving a system $AX = B$ is easy, as we just need to solve for an X such that $RX = Q^{-1}B = Q^{\mathsf{T}}B$, which can be done using back-substitution, as R is upper triangular. This decomposition is called QR factorization.

In [10], a QR factorization scheme is proposed to run the LCMV target classifier in real-time. In particular, a systolic array implementation of QR factorization is proposed, as described by Dick et al. [36].

The proposed systolic array implementation needs a number of resources quadratic on the size of the matrix to invert. The largest matrix on (1) to invert is the correlation matrix, whose dimensions are $L \times L$, where L is the number of spectral bands. For reference, Hydice images are formed by 169 spectral bands [37]. We would need adders, multipliers and square root computers (which are very hardware-intensive) in the order of $169^2 = 28\,561$, which exceeds many FPGAs capabilities.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 05:56:59 UTC from IEEE Xplore. Restrictions apply.

Indeed, in the QR decomposition using systolic arrays [36], only matrices up to dimension 10×10 are tested, using a Xilinx XC7VX690T FPGA. Therefore, for our use case, the systolic array implementation of QR factorization is inadequate.

QR decomposition can also be implemented without a systolic array by using Givens rotations or Householder transformations [43], but they require a costly square root operation, and in terms of computational complexity, they perform asymptotically the same as Gauss–Jordan and LDL decomposition. Therefore, we will not use QR factorization in our hardware implementation.

3) LDL Decomposition: Another popular decomposition for symmetric positive definite matrices is the Cholesky decomposition: every matrix A satisfying those hypotheses can be expressed as $A = \mathrm{LL}^{\mathsf{T}}$, where L is a lower triangular matrix [29].

Unfortunately, the Cholesky decomposition also requires the costly computation of square roots. However, if A is symmetric and positive semidefinite, A can be factored as $A = \mathrm{LDL}^{\mathsf{T}}$, where L is a lower triangular matrix with ones in its diagonal and D is a diagonal matrix [29]. This decomposition does not need to compute any square root. Therefore, if every matrix to invert is symmetric positive semidefinite, then LDL decomposition would be a viable alternative, as the triangular systems could be solved using forward and back-substitution, and the diagonal system could be solved using division.

As R is a correlation matrix, it is symmetric. Moreover, if R is invertible (as we and [10] are assuming, because given that $\vec{r}_i$ have some randomness, the probability of R being invertible is 1, due to the vector subspace where the rows of R are dependent having measure 0), then R is positive definite and its inverse R^{-1} will be positive definite too, so there exists a Cholesky decomposition for $R^{-1} = \mathrm{LL}^{\mathsf{T}}$. Therefore,

$$\begin{aligned} T^{\mathsf{T}}R^{-1}T &= T^{\mathsf{T}}\mathrm{LL}^{\mathsf{T}}T = \left(T^{\mathsf{T}}L\right)\left(L^{\mathsf{T}}T\right) \\ &= \left(T^{\mathsf{T}}L\right)\left(T^{\mathsf{T}}L\right)^{\mathsf{T}}. \end{aligned} \tag{3}$$

As $T^{\mathsf{T}}R^{-1}T$ is expressed as a product of a matrix and its transpose, then $T^{\mathsf{T}}R^{-1}T$ is symmetric and positive semidefinite, and LDL decomposition can be used to solve both linear systems.

One advantage of LDL decomposition over Gaussian elimination is that LDL decomposition requires $(n^3/3)$ operations, around half the operations needed for Gaussian elimination [29]. Let $A = \mathrm{LDL}^{\mathsf{T}}$. Denoting $(D)_{i,i} = d_i$, $(A)_{i,j} = a_{i,j}$ and $(L)_{i,j} = l_{i,j}$, LDL decomposition is given as follows:

$$\begin{cases} d_i = a_{i,i} - \displaystyle\sum_{k=1}^{i-1} l_{i,k}^2 d_k \\ l_{j,i} = \dfrac{1}{d_i}\left(a_{i,j} - \displaystyle\sum_{k=1}^{i-1} l_{j,k} d_k l_{i,k}\right) \quad \text{for } j \geq i+1. \end{cases} \tag{4}$$

C. Fixed-Point Precision Comparison

As seen in Section II-B, both Gauss–Jordan and LDL decomposition can be used to solve linear systems without computing square roots. Another alternative is computing R^{-1} incrementally using the Sherman–Morrison formulas.

One of the most important properties of numerical methods is their accuracy. A hardware-oriented way of measuring accuracy is by determining the minimum number of fixed-point bits needed for the result of the fixed-point computation to be within some tolerance of a golden-model 64-bit floating-point computation.

Hardware operating on fixed-point numbers tends to be less resource-intensive than floating-point hardware, so our initial accuracy analysis will focus on the minimum bit-width needed for accurate fixed-point execution of the LCMV algorithm, comparing different numerical methods.

It has been shown [35] that in some problems, LDL decomposition has better error accuracy than matrix inversion, but we will still perform a more detailed study of our specific use case.

In order to compare the accuracy of the methods, all approaches (target classification using Gauss–Jordan matrix inversion, LDL decomposition, and Sherman–Morrison for computing R^{-1} + LDL decomposition for the second system) have been coded in MATLAB, and they have been converted into fixed-point using MATLAB's Fixed-Point Converter utility.

Performing a binary search on the fixed-point word length, we can find the smallest word length for each numerical method so that the result is within a 2.5% average relative error compared to the output of the 64-bit floating-point reference algorithm. The average relative error is computed as

$$\mathrm{ARE} = \frac{1}{\mathrm{NM}} \sum_{i=1}^{N} \sum_{j=1}^{M} \frac{|O_{i,j} - \hat{O}_{i,j}|}{\max_l \left\{O_{l,j}\right\}}$$

where N is the number of hyperspectral pixels, M is the number of classification classes, $O_{i,j}$ is the 64-bit floating-point output for sample i and classification class j, and $\hat{O}_{i,j}$ is the reduced-precision output for sample i and classification class j.

Please note that each pixel error is normalized relative to the intensity of the brightest pixel (within each classification class), which corresponds to the target detected with the highest strength. This normalization allows us to focus on error magnitudes that could affect detected targets. In contrast, small absolute errors in background noise, which might be associated with a big relative error for that specific pixel, are not as important for classification.

The resulting fixed-point word lengths for a typical image such as the Hydice scene described in Section IV can be seen in Table I. We can see that for LCMV classification, Gauss–Jordan inversion needs 25% more fixed-point bits than LDL decomposition, which translates into both frequency and resource overhead. Moreover, the approach using Sherman–Morrison for the computation of R^{-1} and LDL decomposition for solving the second system needs more than 62-bit fixed-point bits, which is the limit of MATLAB's Fixed-Point Converter utility in our algorithm. The hardware design has many large buses, so reducing the number of bits of each element substantially affects routing congestion and LDL decomposition emerges as an interesting choice.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 05:56:59 UTC from IEEE Xplore. Restrictions apply.

TABLE I
PRECISION COMPARISON OF LINEAR SOLVERS

Linear solve method	Fixed-Point word length
Gauss-Jordan inversion	59 bits
LDL decomposition	47 bits
Sherman-Morrison + LDL	> 62 bits

Note that even LDL decomposition needs 47 bits for a fixed-point implementation, and different submodules would need different scalings, greatly complicating the design, requiring shifting between submodules, and negating the advantages of using a fixed-point representation. Therefore, in our hardware implementation, we have chosen to use a smaller 32-bit floating-point representation to reduce routing congestion and overall resource usage, which has an 8.47% average relative error compared to the 64-bit floating-point output. By performing thresholding with the details described in Section IV-B, the average relative error of 32-bit floating-point with respect to 64-bit floating-point is reduced to 2.06%.

Nonetheless, the fixed-point word length analysis is useful on its own to empirically explore the accuracy and stability of different linear solver methods for the LCMV classification problem. In Section IV-B, we further explore the accuracy of LDL decomposition with fixed-point and floating-point representations.

D. LCMV Algorithm Pseudocode

In this section, we introduce the pseudocode that implements the equations previously exposed and serves as the foundation for the hardware implementation. The pseudocode also describes what operations are performed in parallel according to the resources available in the datapath of the hardware accelerator described in Section III. All dot products (denoted as $\langle \cdot, \cdot \rangle$), element-wise vector multiplications (denoted as $\odot$), vector additions, and scalar-vector multiplications are parallelized.

1) Correlation Matrix Computation: The correlation matrix of a given sequence of samples $\{\vec{r}_1, \vec{r}_2, \ldots, \vec{r}_N\}$ is computed incrementally to adapt the hardware implementation to the streaming nature of hyperspectral sensors. $R = (1/N)\sum_{i=1}^{N} \vec{r}_i \vec{r}_i^{\,\mathsf{T}}$, and expanding coordinates

$$R = \frac{1}{N} \begin{pmatrix} | & | & \cdots & | \\ \vec{r}_1 & \vec{r}_2 & \cdots & \vec{r}_N \\ | & | & \cdots & | \end{pmatrix} \begin{pmatrix} — & \vec{r}_1 & — \\ — & \vec{r}_2 & — \\ \vdots & \vdots & \vdots \\ — & \vec{r}_N & — \end{pmatrix}.$$

Therefore, $R_{i,j} = (1/N)\sum_{k=1}^{N} (\vec{r}_k)_i (\vec{r}_k)_j$ and the jth row of R is

$$R_{(j,:)} = \frac{1}{N} \sum_{k=1}^{N} \vec{r}_k \left(\vec{r}_k\right)_j. \tag{5}$$

Algorithm 1 implements (5) to compute the correlation matrix incrementally.

2) LDL Decomposition: Algorithm 2 performs LDL decomposition in place following (4). Performing the decomposition in place helps reduce memory usage.

Algorithm 1 Correlation Matrix Computation

Input: $X = \{\vec{r}_1, \vec{r}_2, \ldots, \vec{r}_N\}$, with $\vec{r}_i \in \mathbb{R}^L$
Output: $R \in \mathcal{M}_{L,L}(\mathbb{R})$
  local var $\vec{d} \in \mathbb{R}^L$
  for $\vec{r}_i \in X$ **do** ▷ For each hyperspectral sample
    $\vec{d} \leftarrow \vec{r}_i$
    for $j \leftarrow 0 \,..\, L-1$ **do**
      if $i = 0$ **then**
        $\vec{t} \leftarrow \vec{0}$
      else
        $\vec{t} \leftarrow R_{j,:}$
      end if
      $R_{j,:} \leftarrow \vec{d} \cdot d_j + \vec{t}$
    end for
  end for
  for $i \leftarrow 0 \,..\, L-1$ **do** ▷ Divide by N matrix R
    for $j \leftarrow 0 \,..\, L-1$ **do**
      $R_{i,j} \leftarrow R_{i,j}/N$
    end for
  end for

Algorithm 2 LDL Decomposition

Input: $A \in \mathcal{M}_{n,n}(\mathbb{R})$
Output: $L \in \mathcal{M}_{n,n}(\mathbb{R}), \vec{d} \in \mathbb{R}^n$ ▷ L is stored in the lower triangular part of A
  local var $v \in \mathbb{R}^n$
  for $j \leftarrow 0 \,..\, n-1$ **do**
    $v_{1:j-1} \leftarrow A_{j,1:j-1} \odot d_{1:j-1}$
    $d_j \leftarrow A_{j,j} - \langle A_{j,1:j-1}, v_{1:j-1} \rangle$
    for $k \leftarrow j+1 \,..\, n-1$ **do**
      $A_{k,j} \leftarrow \frac{A_{k,j} - \langle A_{k,1:j-1}, v_{1:j-1} \rangle}{d_j}$
    end for
  end for

3) Back-Substitution: To solve a linear system, LDL decomposition as described in Section II-B3 is used to decompose $A = \mathrm{LDL}^{\mathsf{T}}$. Once L and D are computed by Algorithm 2, the linear systems involving triangular matrices L and L^{T} must be solved.

Algorithm 3 uses back-substitution in order to solve the triangular systems. As D is a diagonal matrix, solving $\mathrm{DX} = B$ involves only dividing each row of B by an element of the diagonal of D.

4) LCMV Classifier: Finally, all the functions studied above can be combined to compute the optimum weights for the LCMV target classifier, as can be seen in Algorithm 4.

III. HARDWARE IMPLEMENTATION

The LCMV target classifier is composed of several submodules that implement the algorithms described in Section II-D. The main module diagram can be seen in Fig. 1.

The main module instantiates all the shared common data storage modules, such as matrices, and only the ports to access those matrices are passed as inputs to each submodule. In case arbitration is needed, it is performed by the control unit. Costly shared submodules such as vector ALUs are also

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 05:56:59 UTC from IEEE Xplore. Restrictions apply.

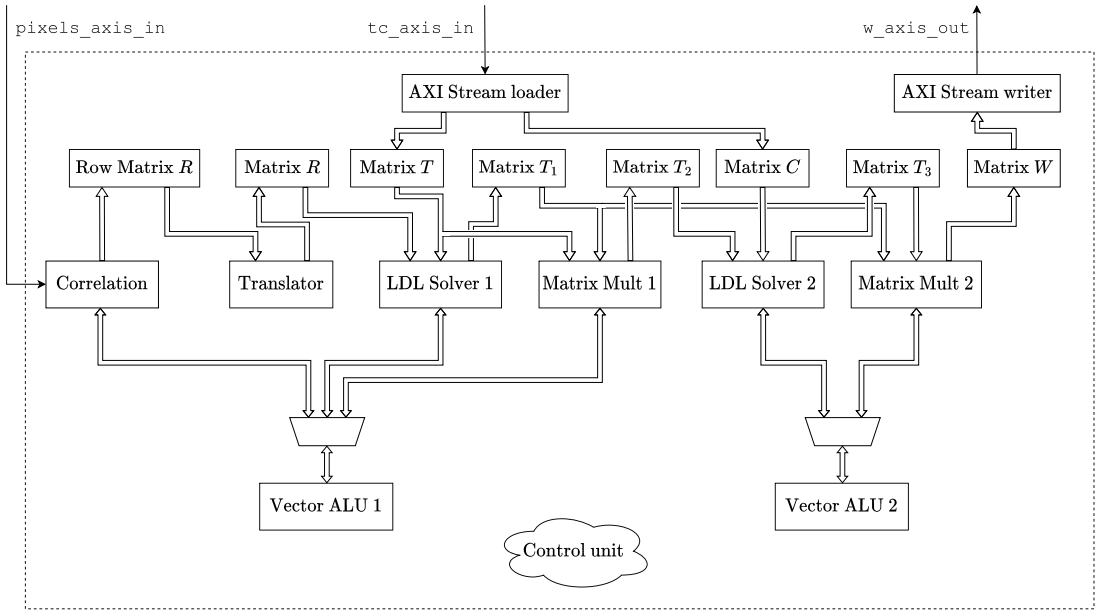


Fig. 1. Main module outline.

Algorithm 3 Triangular Linear System Solver

Input: $A \in \mathcal{M}_{n,n}(\mathbb{R})$, $B \in \mathcal{M}_{n,m}(\mathbb{R})$, $is_lower_tri \in \{\texttt{true}, \texttt{false}\}$
Output: $X \in \mathcal{M}_{n,m}(\mathbb{R})$

▷ L is always stored in the lower triangular part of A. If is_lower_tri is true, the function will solve $LX = B$, otherwise, it will solve $L^{\mathsf{T}}X = B$.

▷ The diagonal of L is assumed to be all ones, which is the case for LDL decomposition.

if is_lower_tri **then**
  $range \leftarrow 0\,..\,n-1$
else
  $range \leftarrow n-1\,..\,0$
end if
for $i \leftarrow range$ **do**
  for $k \leftarrow 0\,..\,m-1$ **do**
    $X_{i,k} \leftarrow B_{i,k}$
    if is_lower_tri **then**
      $X_{i,k} \leftarrow B_{i,k} - \langle A_{i,0:i-1}, X_{0:i-1,k} \rangle$
    else
      $X_{i,k} \leftarrow B_{i,k} - \langle A_{i+1:n-1,i}, X_{i+1:n-1,k} \rangle$
    end if
  end for
end for

Algorithm 4 LCMV Classifier Weight Computation

Input: $X = \{\vec{r}_1, \ldots, \vec{r}_N\}$, with $\vec{r}_i \in \mathbb{R}^L$, $T \in \mathcal{M}_{L,K}(\mathbb{R})$, $C \in \mathcal{M}_{K,M}(\mathbb{R})$
Output: $W^* \in \mathcal{M}_{L,M}(\mathbb{R})$ optimal LCMV weights for target classification.

local var $R \in \mathcal{M}_{L,L}(\mathbb{R})$
$R \leftarrow \texttt{correlation_matrix}(X)$
local var $T_1 \in \mathcal{M}_{L,K}(\mathbb{R})$
$T_1 \leftarrow \texttt{ldl_solve}(R, T)$ ▷ Solve $RT_1 = T$
local var $T_2 \in \mathcal{M}_{K,K}(\mathbb{R})$
$T_2 \leftarrow \texttt{matrix_mult}(T^{\mathsf{T}}, T_1)$ ▷ $T_2 = T^{\mathsf{T}}T_1$
local var $T_3 \in \mathcal{M}_{K,M}(\mathbb{R})$
$T_3 \leftarrow \texttt{ldl_solve}(T_2, C)$ ▷ Solve $T_2T_3 = C$
$W^* \leftarrow \texttt{matrix_mult}(T_1, T_3)$ ▷ $W^* = T_1T_3$

instantiated in the main module, and their ports are passed to the subcomponents that need to use them.

There may be two different instances of the same component with different generic parameters. For example, there are two vector ALUs, as they operate on vectors of different widths, and there are two LDL solver controllers, operating on matrices of different sizes. These modules are duplicated for easier FPGA routing.

All the datapaths are deeply pipelined, outputting a new element each cycle. The pipelining strategy we employed consists in iteratively splitting the critical path with pipeline registers to achieve higher frequencies. In particular, all arithmetic operations are pipelined and memory outputs are also registered. Each module parallelizes a dimension of its inputs if possible, usually in the number of hyperspectral bands (169 for Hydice and 224 for AVIRIS). For synchronizing the datapaths, some modules use finite state machines, and other modules utilize counters and delay registers.

All inputs and outputs of the main module are AXI-Stream ports, which allow for easy integration into an embedded system. The input requires the pixels to be arranged in band interleaved by pixel (BIP) order to allow for the incremental correlation computation shown in Section III-C. This is not an unrealistic requirement since sensors usually work in a pushbroom or whiskbroom fashion, already outputting data in BIP or band interleaved by line (BIL) mode. At most, a line buffer (of size pixels per line times the number of bands) would be required to convert from BIL to BIP.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 05:56:59 UTC from IEEE Xplore. Restrictions apply.

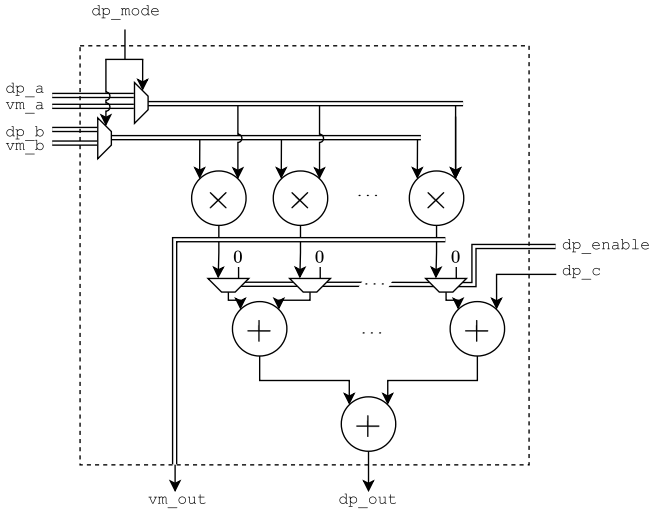


Fig. 2. Vector ALU implementation.

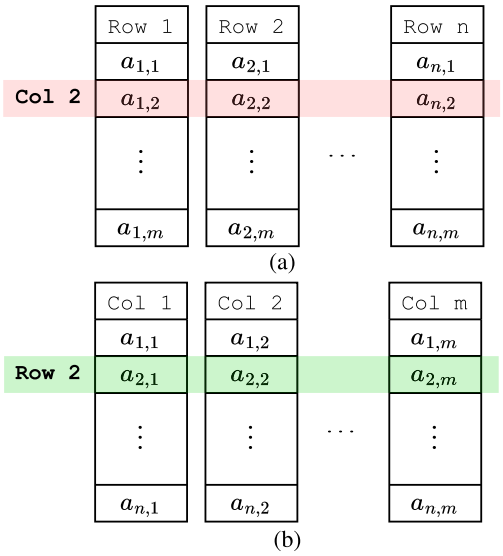


Fig. 3. Matrix storage using duplicated values in BRAMs. (a) Row BRAMs. (b) Column BRAMs.

A. Vector ALU

The biggest computational element in the datapath area-wise is the vector ALU, whose diagram can be seen in Fig. 2. The vector ALU is able to perform either an element-wise product or the dot product of two vectors, computed using an adder tree. For the dot product, the ALU accepts an additional bit vector input `dp_enable`, which determines what vector indices are used for the dot product. An index i will contribute to the final dot product result only if `dp_enable`$[i] = 1$. The vector ALU also accepts a scalar argument `dp_c` to be added to the dot product result using the adder tree, computing $\langle \vec{a}, \vec{b} \rangle + c$.

B. Matrix Storage

In the algorithms described in Section II-D, the need to access both either a full row or a full column of a given matrix arises sometimes. Therefore, matrix data structures that allow us to efficiently retrieve both a whole row and a whole column at the same time are needed.

In order to satisfy those constraints, the stored information of each matrix is duplicated, as can be seen in Fig. 3. To store a matrix $A \in \mathcal{M}_{n,m}(\mathbb{R})$, every element of the matrix is stored twice. On one hand, n BRAMs are instantiated, where the concatenation of the elements at position i at each BRAM forms the ith column of the matrix. On the other hand, m BRAMs are instantiated to be able to read a full row of the matrix in one cycle.

A full row and a full column can be read from a matrix simultaneously, but only a single element can be written at the same time. The element is written twice: to the appropriate row and column BRAMs.

All vendor-specific BRAM and arithmetic core instantiations are wrapped inside a parameterized module that provides a unified interface. Therefore, only a few wrapper files would have to be edited when porting the design to use different vendor-specific IP cores or even open-source ones.

C. Correlation Computation

The datapath shown in Fig. 4 implements correlation computation as described in Algorithm 1. The correlation matrix

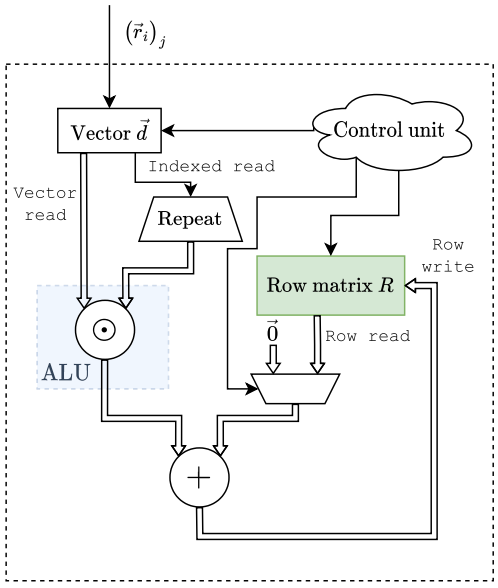


Fig. 4. Correlation computation datapath.

computation module operates pixel by pixel and computes R incrementally. It receives the pixels from an AXI-Stream port. Therefore, the correlation matrix computation is overlapped with the hyperspectral sensor image capture.

The correlation matrix computation algorithm is unique in the way that a full row needs to be written to the correlation matrix R in a single cycle. However, R does not need to be read by columns inside Algorithm 1. Therefore, and only for this module, the matrix R is stored as a single BRAM in which reading the element at index i directly outputs the ith row of R.

A different module later translates this matrix into our duplicated BRAM format, to be efficiently accessed by the LDL decomposition datapath.

D. LDL Decomposition

The datapath for LDL decomposition as described by Algorithm 2 can be seen in Fig. 5. In the LDL decomposition

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 05:56:59 UTC from IEEE Xplore. Restrictions apply.

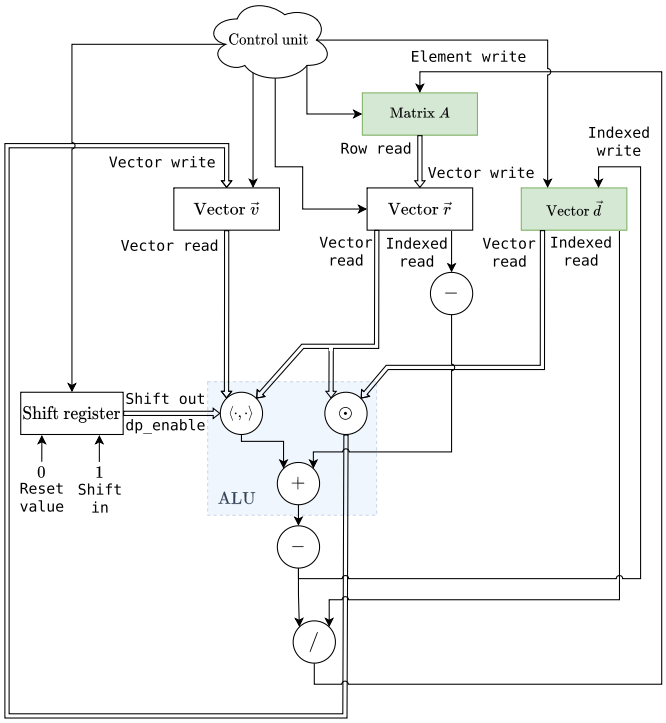


Fig. 5. LDL decomposition datapath.

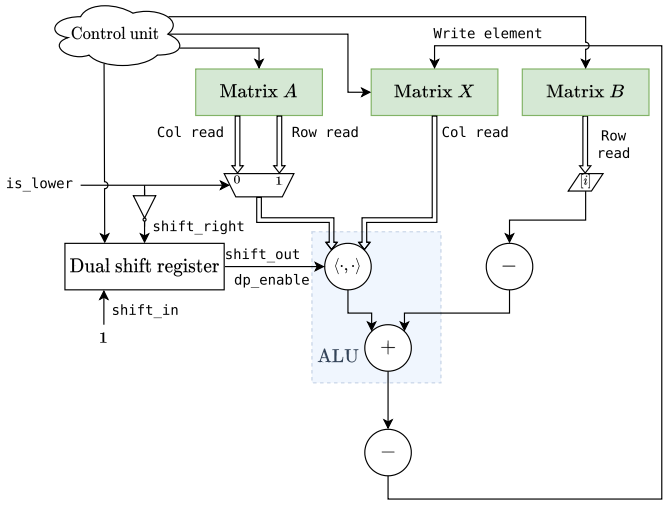


Fig. 6. Triangular system solver datapath.

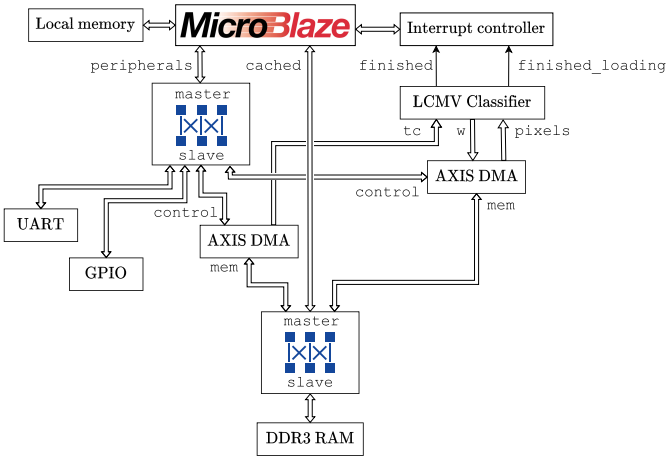


Fig. 7. Evaluation embedded system diagram.

algorithm, the dot product of vector slices needs to be computed, such as $\langle A_{k,1:j-1}, v_{1:j-1}\rangle$, which is done using the `dp_enable` input of the vector ALU. As j is incremented by one in each loop iteration, `dp_enable` can be efficiently controlled by a serial-in parallel-out shift register.

In the algorithm description, $c - \langle \vec{a}, \vec{b}\rangle$ also needs to be computed, but the vector ALU can only compute $\langle \vec{a}, \vec{b}\rangle + c$. In order to compute the subtraction, the sign of both the input c and the dot product output is inverted.

The datapath is fully pipelined, and an iteration of the inner loop can be launched per cycle. To be able to decouple the launching of an iteration with the fulfillment of that iteration by storing the result in memory, the control unit is divided into two state machines. The job of the first state machine is to fill the datapath with data as fast as possible, while the second state machine stores the generated results into the appropriate indices of the matrix.

E. Triangular System Solver

The datapath shown in Fig. 6 implements the triangular system solver described in Algorithm 3. The input `is_lower` controls whether the module solves a lower or upper triangular system. Depending on the value of `is_lower`, the elements enabled in each iteration for the dot product vary. Therefore, instead of controlling `dp_enable` with a simple shift register, a dual shift register is used, which can be shifted to the left or to the right.

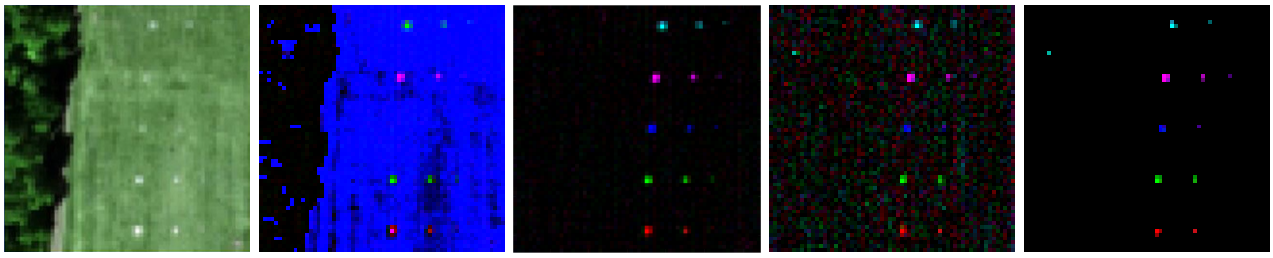
IV. Experimental Results

In this section, results for the FPGA implementation of the LCMV algorithm are presented. First, the experimental setup is described along with the datasets. A brief discussion about floating-point precision follows, justifying the selection of 32-bit floating-point logic. Then, performance metrics are given regarding the hardware accelerator, which is finally compared against software references for COTS CPU, embedded CPU, and embedded GPU. The hardware implementation RTL and software model code can be found in our GitHub repository [44].

A. Experimental Setup

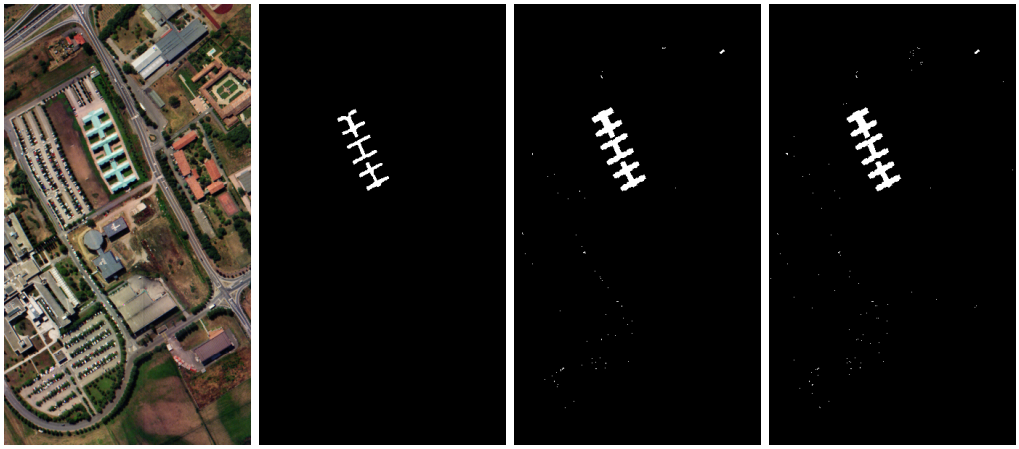
The hardware architecture for the LCMV classifier specified in Section III was written in the SystemVerilog HDL language. Xilinx Vivado 2020.2 [45] and Xilinx Vitis 2020.2 [46] were used to create an embedded system integrating the hardware accelerator, which is illustrated in Fig. 7. The hyperspectral pixels are fed to the hardware accelerator using AXI-Stream DMA, and the output is also written to memory using AXI-Stream DMA. The MicroBlaze processor in the embedded system is used to check the output against a software model and verify the correct functionality. It is important to note that the hardware implementation output is equal bit by bit to the golden software model.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 05:56:59 UTC from IEEE Xplore. Restrictions apply.



(a) (b) (c) (d) (e)

Fig. 8. Hydice scene classification. (a) Hydice scene. (b) Fixed-point 46 bits. (c) Fixed-point 47 bits. (d) Floating-point 32 bits. (e) Floating-point 32 bits + threshold.



(a) (b) (c) (d)

Fig. 9. Pavia University scene detection. (a) Pavia University scene. (b) Ground truth. (c) Floating-point 64 bits + threshold. (d) Floating-point 32 bits + threshold.

The complete system was implemented targeting a Kintex Ultrascale XQRKU060 and Virtex-7 XC7VX690T FPGA. The tests and hardware verification were carried out on a Xilinx VC709 board [47], featuring the XC7VX690T FPGA and two DDR3 SODIMM memory slots, able to hold up to 4 GB each. However, results are reported for the radiation-tolerant XQRKU060 FPGA, as it better suits the potential space applications we are targeting.

For performance and utilization evaluation, we will target the ROSIS [48], Hydice [37], and AVIRIS [38] sensors, configuring the FPGA implementation for each one. The tested configurations targeted 103, 169, and 224 bands, respectively. Pre-processing algorithms may remove some noisy bands, or aggregate highly correlated ones. This results in less time and hardware being required for target detection. We present the results here for the aforementioned number of bands, presenting an upper bound for resource utilization if less bands are used in practice.

For testing and verification of our system, we will use the following datasets.

1) *Hydice scene*: It consists of a 64×64 image captured by the Hydice sensor. It has been studied in several papers [10], [49]. After pre-processing, it contains 169 spectral bands. It consists of 15 panels arranged in a matrix of five rows and three columns. The panels in each row are painted using the same material, and different rows are painted with different materials. Moreover, in each row, the sizes of the panels decrease. The panels, arranged from left to right, measure 3×3 m, 2×2 m, and 1×1 m, respectively. The RGB-colored image can be seen in Fig. 8(a).
2) *Pavia University scene* [50]: It consists of a 340×610 image captured by the ROSIS sensor over Pavia University in Italy and contains 103 spectral bands. We use this dataset mainly for the detection of the painted metal sheets on the roof. The RGB-colored image can be seen in Fig. 9(a), and the ground truth of the metal sheets is shown in Fig. 9(b).

B. Accuracy Evaluation

In Section II-C, it was demonstrated that a total of 47 bits is necessary for the fixed-point implementation of LDL decomposition to achieve satisfactory accuracy. This is evident in Fig. 8(b) and 8(c), which illustrate the classification results for the Hydice scene with 46 and 47 fixed-point bits, with average relative errors of 79.34% and 0.01%, respectively.

Fig. 8(d) reveals that the utilization of 32-bit floating-point introduces some noise in the background. Nevertheless, the application of simple thresholding effectively eliminates most of the noise, as depicted in Fig. 8(e). Thresholding was applied manually, setting an intensity value T_h below which all outputs were set to zero, which reduces the average relative error of the 32-bit floating-point Hydice output from 8.47% to 2.06%.

Thresholding proves to be a valuable technique for simple denoising and enables the use of floating-point representations

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 05:56:59 UTC from IEEE Xplore. Restrictions apply.

TABLE II
UTILIZATION, FREQUENCY, AND POWER SUMMARY

	XC7VX690T			XQRKU060		
	ROSIS	Hydice	AVIRIS	ROSIS	Hydice	AVIRIS
Bands	103	169	224	103	169	224
BRAM Tiles	173.5 (11.8%)	269 (18.3%)	349 (23.71%)	46 (4.26%)	269 (24.9%)	348.5 (32.3%)
DSP48E1s	944 (26.22%)	1472 (40.9%)	1912 (53.11%)	944 (34.2%)	1472 (53.3%)	1912 (69.28%)
Slice Registers	168500 (19.45%)	251756 (29.1%)	321114 (37.1%)	179329 (27.03%)	254366 (38.4%)	319938 (48.3%)
LUTs as Logic	84768 (19.57%)	129440 (29.9%)	166677 (38.5%)	93744 (28.26%)	133004 (40.1%)	175913 (53.0%)
LUTs as Memory	12480 (7.16%)	20076 (11.5%)	25924 (14.9%)	34598 (23.56%)	21788 (14.8%)	28097 (19.1%)
Total Slices	38476 (35.53%)	55769 (51.5%)	192601 (44.5%)	23662 (57.07%)	29410 (70.9%)	38032 (91.7%)
Maximum frequency	190 MHz	165 MHz	160 MHz	166 MHz	153 MHz	144 MHz
Power @ Max Freq	16.31 W	12.21 W	15.63 W	13.6 W	22.3 W	29.53 W

with fewer bits than fixed-point, helping with resource utilization and routing congestion.

In Fig. 9(b)–(d), it is evident that floating-point representation is also well-behaved in the Pavia University scene. It is remarkable that there are minimal differences between the 64-bit and 32-bit floating-point classifications, obtaining an average relative error of 0.04% after thresholding. There is still some residual noise, which could be further mitigated by stipulating a minimum number of detected neighbors for the identified pixels.

C. Performance Evaluation

The utilization of different FPGA resources, the maximum achievable frequency, and power for ROSIS, Hydice, and AVIRIS configurations with 15 targets to be detected and a 3-D output space can be seen in Table II. Note that image size does not impact FPGA utilization or frequency, as the pixels are incrementally fed into the correlation matrix computation module. Nevertheless, the number of spectral channels, the number of desired targets, and the output dimension do impact resource utilization.

Resource utilization is dominated by the number of spectral channels, since the number of targets and output dimension is smaller (around 5–15 targets and three output channels compared to over 100 bands). From Algorithm 4, it follows that correlation computation and the first LDL decomposition are the steps that operate over the biggest matrices, proportional in size to the square of the number of bands.

The maximum achievable frequency for a given number of spectral channels is represented in Fig. 10, which illustrates the reduction in frequency as the number of spectral channels increases, eventually stabilizing around 160 MHz for the XC7VX690T FPGA, and 140 MHz for the XQRKU060 FPGA. The main factor affecting frequency is routing congestion. With a large number of spectral channels, huge buses must be routed across the FPGA, causing global routing congestion. Therefore, frequency could be further improved by serializing some buses, at the cost of hardware complexity and added latency.

The number of cycles spent during each LCMV classification step is shown in Fig. 11. LDL1 and Mult1 correspond to the first `ldl_solve` and `matrix_mult` calls in Algorithm 4, while LDL2 and Mult2 correspond to the second ones.

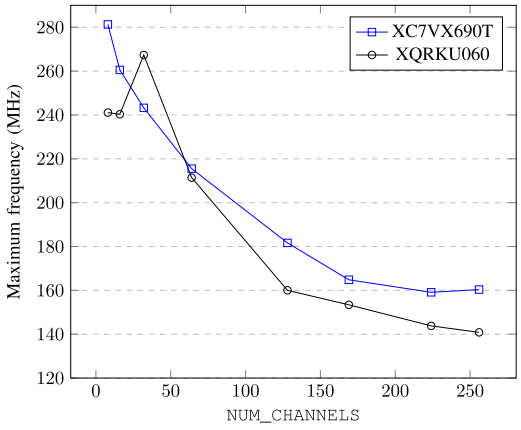


Fig. 10. Maximum achievable frequency.

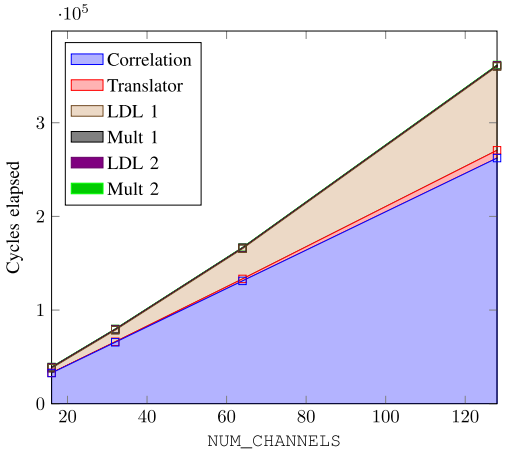


Fig. 11. Itemized elapsed clock cycles.

The majority of the time is spent computing the correlation matrix (which can be overlapped with the hyperspectral image capture) and its LDL decomposition. As the number of spectral channels increases, the LDL decomposition accounts for a larger portion of the total time, given that the parallelized LDL decomposition incurs a time complexity of $\mathcal{O}(L^2)$, whereas the correlation matrix computation complexity is $\mathcal{O}(L)$.

1) Real-time processing: The processing time for the Hydice scene is broken down in Table III, totaling 10.094 ms for the XQRKU060 FPGA.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 05:56:59 UTC from IEEE Xplore. Restrictions apply.

TABLE III
ELAPSED TIME (MS) PER STEP, TOTAL TIME, AND SPEEDUP. CAPTURE TIME IS 153.4MS

Step	Hardware (XQRKU060)	Jetson CPU (SIMD)	Jetson GPU (LDL)	Jetson GPU (Cholesky)	MATLAB (COTS)
Data load	N/A	N/A	2.447	2.447	N/A
Correlation	9.049	836.84	1.669	1.669	2.179
Translator	0.094	0	0.026	0.026	0.106
LDL 1	0.919	24.92	25.361	2.663	0.955
Mult 1	0.002	0.271	0.053	0.053	0.031
LDL 2	0.026	0.053	0.724	0.630	0.058
Mult 2	0.004	0.054	0.043	0.043	0.020
Data offload	N/A	N/A	0.05	0.05	N/A
Total	**10.094**	**862.138**	**30.373**	**7.581**	**3.349**
Total (Since last pixel)	1.045	862.138	30.373	7.581	3.349
Speedup (Since last pixel)	1	825	29.1	7.25	3.20

The Hydice sensor takes between 12 and 22 ms to capture a full row of 320 pixels [37]. Therefore, the sensor can capture between 14 545 and 26 667 hyperspectral pixels per second.

Considering the fastest case, 26 667 pixels per second, the Hydice sensor can capture the 64×64 Hydice scene image in $(64 \cdot 64/26\,667) \approx 153.4$ ms (4.51 MS/s). As our hardware module only takes 10.093 ms $<$ 153.4 ms for the XQRKU060 FPGA, our proposed implementation operates in real-time, exceeding by $15\times$ the real-time performance requirements (reaching 69 MS/s). Moreover, as seen in Section III-C, the correlation hardware module operates incrementally on each newly received pixel and does not need to wait until the sensor has collected the whole image. Therefore, most of the correlation computation latency can be hidden behind the sensor's latency for taking the image. This idea is further explored in Section IV-D.

To provide a general idea of its performance, the total number of cycles C required by our implementation to process an image is approximated by the equation $C \approx 2NL+5L^2+E$, where N is the number of pixels, L is the number of bands, and E is a small value proportional to LKM (K being the number of targets and M the output dimension). This formula is obtained by analyzing the iteration count of the loops in Section II-D and adjusting the coefficients to fit the number of cycles elapsed for different configurations.

Using this formula to compute the total number of cycles, and considering the clock frequency F in Table II, we calculate the throughput as $F \cdot S/C$, where $S = N \cdot L$ is the number of samples of the processed image. Our XQRKU060 implementation can achieve a maximum throughput of $\approx$ 71.88 MS/s for AVIRIS (Pavia scene), well-exceeding the real-time constraints [51] at 1.83 MS/s, and around 69 MS/s for Hydice (Hydice scene), which has a real-time requirement of 4.51 MS/s.

Newer sensors such as AVIRIS-NG [52] or the NACHOS sensor [53] are, respectively, capable of running at 37.5 and 12.25 MS/s. The case of the NACHOS sensor is of special interest since it is deployed onboard a small cubesat satellite, where the XQRKU060 can mitigate radiation-induced errors. In principle, their real-time throughput requirements are lower than the throughputs achieved by our design for Hydice and AVIRIS. However, our design can only be implemented with up to 256 bands on the XQRKU060 FPGA due to resource limitations, while the number of bands of AVIRIS-NG and NACHOS is, respectively, 425 and 350.

In these cases, we assume the application of dimensionality reduction techniques prior to classification, which can reduce the number of bands without greatly impacting accuracy. An example is band selection [54], which maintains classification accuracy even when reducing the number of bands by a factor of $10\times$. In any case, the prior dimensionality reduction would not need to be that aggressive in our case, since a core processing up to 256 bands has been successfully implemented.

D. Software Comparison

In this section, a comparison will be made between the performance of our proposed hardware implementation and several software implementations.

We have compared our hardware implementation against two different systems.

1) **COTS CPU**: An AMD Ryzen 9 5900HX 45W 3.3 GHz (boost up to 4.6 GHz) CPU system with eight cores and 16 threads, 32 GB RAM on Windows 64-bit, with all optimization settings enabled.
2) **Embedded CPU and GPU**: An Nvidia Jetson Orin Nano board with 1024 CUDA cores, 32 tensor cores, six ARM cores and 8 GB of RAM. The GPU and CPU run at 625 MHz and 1.5 GHz, respectively. For all of our experiments, it has been run on the 15 W power mode.

The following software implementations have been benchmarked.

1) **MATLAB (COTS)**: A MATLAB implementation that calls the native MATLAB functions for computing the correlation matrix, computing LDL decompositions, solving triangular systems, and multiplying matrices. Our MATLAB version uses Intel oneAPI Math Kernel Library (MKL) [55], which implements highly optimized SIMD linear algebra routines. Compared to a C++ implementation with no library calls, this version is around $80\times$ faster. This MATLAB implementation runs in the COTS CPU.
2) **Jetson CPU**: A C SIMD-optimized version generated from the MATLAB code using the MATLAB *Coder*

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 05:56:59 UTC from IEEE Xplore. Restrictions apply.

package compiled and executed on the Jetson Nano's ARM cores.

3) **Jetson GPU (LDL)**: A port of the MATLAB code to CUDA executed on the Jetson Nano embedded GPU. The CUDA code has been generated using the MATLAB *GPU Coder* package, which created CUDA kernels for performing LDL decomposition, as well as for the other algorithm steps. By using the Jetson GPU and taking advantage of the problem's parallelism, the embedded GPU version greatly outperforms the embedded CPU implementation.
4) **Jetson GPU (Cholesky)**: A port of the MATLAB code to CUDA that uses Cholesky decomposition instead of LDL decomposition. When doing so, MATLAB *GPU Coder* does not generate CUDA kernels for the Cholesky decomposition but calls the highly optimized cuBLAS [56] linear algebra routines, greatly improving on the Jetson GPU LDL implementation. For our use-case, the Cholesky decomposition using square roots is numerically equivalent to the LDL decomposition, obtaining an average relative error of $4 \cdot 10^{-12}\%$ for the Hydice image with respect to the LDL decomposition output.

The performance has been evaluated on a 64×64 image on the LCMV classification module with 169 spectral bands, 15 target signatures, and a 3-D classification output, the Hydice sensor configuration. The broken-down elapsed time for each implementation can be found in Table III.

1) Incremental correlation matrix computation effects: It can be seen that, generally, our hardware implementation is faster than the software counterparts with the exception of correlation computation using MATLAB or the Jetson GPU cuBLAS matrix multiplication library routines.

This is due to the fact that MATLAB already has the full pixel matrix stored in memory, so MATLAB can use more efficient algorithms for matrix multiplication than the naive $\mathcal{O}(n^3)$. For example, Coppersmith–Winograd found an $\mathcal{O}(n^{2.376})$ algorithm to multiply matrices [57], which is asymptotically faster than the naive way.

The same is true for the GPU implementation, where the high CUDA core count essentially brings the cost of matrix multiplication from $\mathcal{O}(n^3)$ down to $\mathcal{O}(n^2)$, while having a clock running at 625 MHz.

However, an important fact about those techniques is that they require having the full pixel matrix already stored in memory, so they cannot be performed incrementally as the pixels arrive.

Our hardware implementation performs the correlation computation incrementally, allowing it to hide the correlation computation latency under the sensor's latency to capture an image, which, as we saw in Section IV-C, can be quite large. Therefore, although the computation of the correlation matrix may take more time in our hardware implementation, this time can be hidden behind the sensor image capturing time, as illustrated in Fig. 12.

Only counting the elapsed time since the last hyperspectral pixel is captured, we obtain a $3.2\times$ speedup with respect to the COTS MATLAB implementation, a $7.25\times$ speedup with respect to the fastest embedded GPU implementation, and an $825\times$ speedup with respect to the SIMD-accelerated embedded CPU.

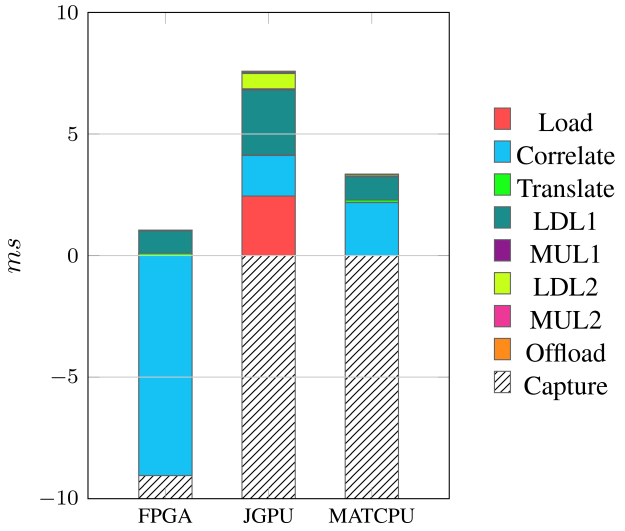


Fig. 12. Detail of time taken by the different steps of the algorithm for the fastest implementations. Capture starts at −153.4 ms. Correlation and capture overlap on the FPGA.

V. Conclusion

A novel FPGA implementation of the LCMV target classification algorithm is presented in this article, based on the idea of exploring parallelizable variations of the algorithm using different numerical methods for solving linear systems with the aim of reaching real-time processing while maintaining the highest possible accuracy, all within the resource constraints of radiation-tolerant FPGAs.

First, several numerical methods for solving the linear systems required for LCMV classification are explored, including the reduction of the LCMV classification problem to LDL decomposition. Special care is taken to prove that the matrices are both symmetric and positive semidefinite, characteristics that are both required by LDL decomposition.

Then, a fixed-point precision analysis is conducted to empirically compare the accuracy of different numerical methods (Gauss–Jordan matrix inversion, Sherman–Morrison incremental matrix inversion, and LDL decomposition). This analysis, based on the final LCMV classification results, reveals that LDL decomposition requires a 25% smaller bitwidth than the other methods while maintaining the same level of accuracy.

LDL decomposition is chosen as the most suitable option for our goals, and the LCMV and LDL algorithms are adapted and parallelized for an efficient FPGA implementation. Special hardware data structures are designed for matrix storage, to meet the high bandwidth demands of our extensively pipelined and parallelized implementation.

An incremental correlation matrix computation is selected for the first phase of the algorithm, which works directly with the streaming data from the sensor, hiding away the most costly part of the algorithm behind the image capture time.

In the end, the core is implemented on a radiation-tolerant Kintex UltraScale XQRKU060 FPGA, where despite finding

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 05:56:59 UTC from IEEE Xplore. Restrictions apply.

some routing congestion due to the bus width, the design still reaches real-time performance for hyperspectral sensors with hundreds of bands such as Hydice and AVIRIS, operating 15× faster than the Hydice sensor real-time requirements. The design has been successfully tested with up to 256 bands on the XQRKU060. For a higher band count, dimensionality reduction or band selection techniques (e.g., removing noisy bands) would be necessary for the design to fit in this board.

The algorithm is also implemented in C++ and CUDA for comparison purposes, and executed on a Jetson Nano Orin using both the embedded CPU and GPU. The embedded CPU fails to meet real-time requirements, running over 6× slower than the Hydice sensor, while the embedded GPU is able to easily outperform the CPU version and achieve speeds 20× faster than real-time. The MATLAB code, running on the COTS AMD Ryzen 9 5900HX processor and utilizing highly optimized SIMD code, is even faster, reaching speeds 45× faster than real-time.

While the embedded GPU, FPGA, and COTS CPU approaches all achieve real-time performance, incrementally calculating the correlation matrix in the FPGA as the image is captured offers two significant advantages: it eliminates the need for external memory to store the image, and it reduces the response time from capturing the last pixel to obtaining results to just 1 ms. This FPGA response time is 3.2×, 7.25×, and 825× faster than the COTS CPU, embedded GPU and embedded CPU, respectively.

Additionally, both the FPGA and GPU are considered embedded devices with reasonable power consumption (respectively, 2× and 3× lower than the COTS CPU), rendering them more suitable for an on-board implementation. The XQRKU060 FPGA also has the advantage of being radiation-tolerant, a necessary requirement for satellite applications.

In the end, the FPGA implementation is found suitable for the initial objective of enabling real-time execution of the LCMV algorithm for aerial and space platforms. GPUs are also found to be suitable when radiation tolerance is not a requirement, making both platforms attractive depending on the application requirements. However, CPU implementations would not achieve real-time performance unless a powerful chip was used, which might be impractical in these scenarios.

This article represents (to the best of the authors' knowledge) the first time that an LDL-based LCMV algorithm has been implemented on an FPGA and a GPU in real-time. This opens up the possibility of bringing target classification to remote sensing devices with low power consumption while maintaining performance and accuracy requirements.

REFERENCES

[1] G. A. Shaw and H. K. Burke, "Spectral imaging for remote sensing," *Lincoln Lab. J.*, vol. 14, no. 1, pp. 3–28, 2003.

[2] J. M. Bioucas-Dias, A. Plaza, G. Camps-Valls, P. Scheunders, N. Nasrabadi, and J. Chanussot, "Hyperspectral remote sensing data analysis and future challenges," *IEEE Geosci. Remote Sens. Mag. Replaces Newsletter*, vol. 1, no. 2, pp. 6–36, Jun. 2013.

[3] C.-I. Chang, "Hyperspectral target detection: Hypothesis testing, signal-to-noise ratio, and spectral angle theories," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, 2022, Art. no. 5505223.

[4] M. Ahmad et al., "Hyperspectral image classification—Traditional to deep models: A survey for future prospects," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 15, pp. 968–999, 2022.

[5] Sneha and A. Kaul, "Hyperspectral imaging and target detection algorithms: A review," *Multimedia Tools Appl.*, vol. 81, no. 30, pp. 44141–44206, Dec. 2022.

[6] G. Tejasree and L. Agilandeeswari, "An extensive review of hyperspectral image classification and prediction: Techniques and challenges," *Multimedia Tools Appl.*, pp. 1–98, Mar. 2024, doi: 10.1007/s11042-024-18562-9.

[7] X. Liu and C. Yang, "A kernel spectral angle mapper algorithm for remote sensing image classification," in *Proc. 6th Int. Congr. Image Signal Process. (CISP)*, vol. 2, Dec. 2013, pp. 814–818.

[8] P. S. Sisodia, V. Tiwari, and A. Kumar, "Analysis of supervised maximum likelihood classification for remote sensing image," in *Proc. Int. Conf. Recent Adv. Innov. Eng. (ICRAIE-)*, May 2014, pp. 1–4.

[9] Q. Du and C.-I. Chang, "A linear constrained distance-based discriminant analysis for hyperspectral image classification," *Pattern Recognit.*, vol. 34, no. 2, pp. 361–373, Feb. 2001.

[10] C.-I. Chang, H. Ren, and S.-S. Chiang, "Real-time processing algorithms for target detection and classification in hyperspectral imagery," *IEEE Trans. Geosci. Remote Sens.*, vol. 39, no. 4, pp. 760–768, Apr. 2001.

[11] Y. Guo, X. Yin, X. Zhao, D. Yang, and Y. Bai, "Hyperspectral image classification with SVM and guided filter," *EURASIP J. Wireless Commun. Netw.*, vol. 2019, no. 1, p. 56, Dec. 2019.

[12] B. Manifold, S. Men, R. Hu, and D. Fu, "A versatile deep learning architecture for classification and label-free prediction of hyperspectral images," *Nature Mach. Intell.*, vol. 3, no. 4, pp. 306–315, Mar. 2021.

[13] S. Kumar, C. Desmedt, D. Larsimont, C. Sotiriou, and E. Goormaghtigh, "Change in the microenvironment of breast cancer studied by FTIR imaging," *Analyst*, vol. 138, no. 14, p. 4058, 2013.

[14] S. Peyghambari and Y. Zhang, "Hyperspectral remote sensing in lithological mapping, mineral exploration, and environmental geology: An updated review," *J. Appl. Remote Sens.*, vol. 15, no. 3, Jul. 2021, Art. no. 031501.

[15] C. Goyens, H. Lavigne, A. Dille, and H. Vervaeren, "Using hyperspectral remote sensing to monitor water quality in drinking water reservoirs," *Remote Sens.*, vol. 14, no. 21, p. 5607, Nov. 2022.

[16] B. Lu, P. Dao, J. Liu, Y. He, and J. Shang, "Recent advances of hyperspectral imaging technology and applications in agriculture," *Remote Sens.*, vol. 12, no. 16, p. 2659, Aug. 2020.

[17] P. Horstrand, R. Guerra, A. Rodríguez, M. Díaz, S. López, and J. Fco. López, "A UAV platform based on a hyperspectral sensor for image capturing and on-board processing," *IEEE Access*, vol. 7, pp. 66919–66938, 2019.

[18] S.-E. Qian, "Hyperspectral satellites, evolution, and development history," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 14, pp. 7032–7056, 2021.

[19] K. Thangavel et al., "Autonomous satellite wildfire detection using hyperspectral imagery and neural networks: A case study on Australian wildfire," *Remote Sens.*, vol. 15, p. 720, Jan. 2023.

[20] M. Shimoni, R. Haelterman, and C. Perneel, "Hypersectral imaging for military and security applications: Combining myriad processing and sensing techniques," *IEEE Geosci. Remote Sens. Mag. Replaces Newsletter*, vol. 7, no. 2, pp. 101–117, Jun. 2019.

[21] V. C. Coffey, "Hyperspectral imaging for safety and security," *Opt. Photon. News*, vol. 26, no. 10, p. 26, 2015.

[22] A. Yusuf and S. Alawneh, "A survey of GPU implementations for hyperspectral image classification in remote sensing," *Can. J. Remote Sens.*, vol. 44, no. 5, pp. 532–550, Sep. 2018.

[23] A. Altamimi and B. Ben Youssef, "A systematic review of hardware-accelerated compression of remotely sensed hyperspectral images," *Sensors*, vol. 22, no. 1, p. 263, Dec. 2021.

[24] S. Lopez, T. Vladimirova, C. Gonzalez, J. Resano, D. Mozos, and A. Plaza, "The promise of reconfigurable computing for hyperspectral imaging onboard systems: A review and trends," *Proc. IEEE*, vol. 101, no. 3, pp. 698–722, Mar. 2013.

[25] D. Gyaneshwar and R. R. Nidamanuri, "A real-time FPGA accelerated stream processing for hyperspectral image classification," *Geocarto Int.*, vol. 37, no. 1, pp. 52–69, Jan. 2022.

[26] J. Lei, G. Yang, W. Xie, Y. Li, and X. Jia, "A low-complexity hyperspectral anomaly detection algorithm and its FPGA implementation," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 14, pp. 907–921, 2021.

[27] R. Macias, S. Bernabé, D. Báscones, and C. González, "FPGA implementation of a hardware optimized automatic target detection and classification algorithm for hyperspectral image analysis," *IEEE Geosci. Remote Sens. Lett.*, vol. 19, pp. 1–5, 2022.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 05:56:59 UTC from IEEE Xplore. Restrictions apply.

[28] *XQRKU060 FPGA*. Accessed: Jun. 10, 2024. [Online]. Available: https://docs.amd.com/v/u/en-U.S./ds882-xqr-kintex-ultrascale
[29] G. H. Golub and C. F. Van Loan, *Matrix Computations*, 3rd ed., Baltimore, MD, USA: Johns Hopkins Univ. Press, 1996.
[30] Q. Du and R. Nekovei, "Fast real-time onboard processing of hyperspectral imagery for detection and classification," *J. Real-Time Image Process.*, vol. 4, no. 3, pp. 273–286, Dec. 2008.
[31] B. Yang, M. Yang, A. Plaza, L. Gao, and B. Zhang, "Dual-mode FPGA implementation of target and anomaly detection algorithms for real-time hyperspectral imaging," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 8, no. 6, pp. 2950–2961, Jun. 2015.
[32] J. Lei et al., "A deep pipelined implementation of hyperspectral target detection algorithm on FPGA using HLS," *Remote Sens.*, vol. 10, no. 4, p. 516, Mar. 2018.
[33] A. Plaza and C. Chang, *High Performance Computing in Remote Sensing* (Computer and Information Science Series). London, U.K.: Chapman & Hall/CRC, 2007, ch. 16, pp. 379–395.
[34] W. W. Hager, "Updating the inverse of a matrix," *SIAM Rev.*, vol. 31, no. 2, pp. 221–239, Jun. 1989.
[35] C. Ingemarsson and O. Gustafsson, "On fixed-point implementation of symmetric matrix inversion," in *Proc. Eur. Conf. Circuit Theory Design (ECCTD)*, Aug. 2015, pp. 1–4.
[36] C. Dick, F. Harris, M. Pajic, and D. Vuletic, "Real-time QRD-based beamforming on an FPGA platform," in *Proc. Fortieth Asilomar Conf. Signals, Syst. Comput.*, Nov. 2006, pp. 1200–1204.
[37] M. L. Nischan, J. P. Kerekes, J. E. Baum, and R. W. Basedow, "Analysis of hydice noise characteristics and their impact on subpixel object detection," in *Proc. SPIE*, Oct. 1999, pp. 112–123.
[38] Jet Propulsion Laboratory (NASA). *Airborne Visible/Infrared Imaging Spectrometer*. Accessed: Feb. 24, 2024. [Online]. Available: https://aviris.jpl.nasa.gov/
[39] C. Ingemarsson and O. Gustafsson, "Hardware architecture for positive definite matrix inversion based on LDL decomposition and back-substitution," in *Proc. 50th Asilomar Conf. Signals, Syst. Comput.*, Nov. 2016, pp. 859–863.
[40] B. V. Veen and K. Buckley, "Beamforming: A versatile approach to spatial filtering," *IEEE ASSP Mag.*, vol. ASSP-5, no. 2, pp. 4–24, Apr. 1988.
[41] C. González, S. Bernabé, D. Mozos, and A. Plaza, "FPGA implementation of an algorithm for automatically detecting targets in remotely sensed hyperspectral images," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 9, no. 9, pp. 4334–4343, Sep. 2016.
[42] L. N. Trefethen and D. Bau, *Numerical Linear Algebra*. Philadelphia, PA, USA: SIAM, 1997.
[43] Å. Björck, *Numerical Methods for Least Squares Problems*. Philadelphia, PA, USA: Society for Industrial and Applied Mathematics, 1996.
[44] P. Palacios. *Hyperspectral LCMV Classifier*. Accessed: Jun. 12, 2024. [Online]. Available: https://github.com/palmenros/hyperspectral-lcmv-classifier/
[45] *Amd Xilinx Vivado*. Accessed: Feb. 24, 2024. [Online]. Available: https://www.xilinx.com/products/design-tools/vivado.html
[46] *Amd Xilinx Vitis*. Accessed: Feb. 24, 2024. [Online]. Available: https://www.xilinx.com/products/design-tools/vitis.html
[47] *Vc709 Evaluation Board*. Accessed: Feb. 24, 2024. [Online]. Available: https://www.xilinx.com/products/boards-and-kits/dk-v7-vc709-g.html
[48] B. Kunkel, F. Blechinger, R. Lutz, R. Doerffer, H. van der Piepen, and M. Schroder, "ROSIS (reflective optics system imaging spectrometer)—A candidate instrument for polar platform missions," in *Proc. SPIE*, C. S. Bowyer and J. S. Seeley, Eds., vol. 0868, 1988, pp. 134–141.
[49] C.-I. Chang and S.-S. Chiang, "Anomaly detection and classification for hyperspectral imagery," *IEEE Trans. Geosci. Remote Sens.*, vol. 40, no. 6, pp. 1314–1325, Jun. 2002.
[50] *Pavia University Scene*. Accessed: Feb. 20, 2024. [Online]. Available: https://www.ehu.eus/ccwintco/index.php/Hyperspectral_Remote_Sensing_Scenes
[51] R. O. Green et al., "Imaging spectroscopy and the airborne visible/infrared imaging spectrometer (AVIRIS)," *Remote Sens. Environ.*, vol. 65, no. 3, pp. 227–248, 1998.
[52] J. W. Chapman et al., "Spectral and radiometric calibration of the next generation airborne visible infrared spectrometer (AVIRIS-NG)," *Remote Sens.*, vol. 11, no. 18, p. 2129, Sep. 2019.
[53] S. P. Love et al., "Nachos, a cubesat-based high-resolution uv-visible hyperspectral imager for remote sensing of trace gases: System overview and science objectives," in *Proc. SPIE*, vol. 11832, 2021, pp. 85–99.
[54] C. Yu, M. Song, and C.-I. Chang, "Band subset selection for hyperspectral image classification," *Remote Sens.*, vol. 10, no. 1, p. 113, Jan. 2018.
[55] *Intel® Oneapi Math Kernel Library*. Accessed: Feb. 29, 2024. [Online]. Available: https://www.intel.com/content/www/us/en/developer/tools/oneapi/onemkl.html
[56] Nvidia. *Basic Linear Algebra on Nvidia GPUS*. Accessed: Jun. 11, 2024. [Online]. Available: https://developer.nvidia.com/cublas
[57] M. Cenk and M. A. Hasan, "On the arithmetic complexity of strassen-like matrix multiplications," *J. Symbolic Comput.*, vol. 80, pp. 484–501, May 2017.



Pedro Palacios received the bachelor's degree (Hons.) in mathematics and computer science from the Complutense University of Madrid, Madrid, Spain, in 2023. He is currently pursuing the M.Sc. degree in computer science with EPFL, Lausanne, Switzerland.

During the last year of his B.Sc., he worked with the Department of Computer Architecture and Automatics, Complutense University of Madrid, through the Research Collaboration Fellowship. He is with a "La Caixa" Foundation Postgraduate Fellowship. His research interests include computer architecture, distributed systems, reconfigurable computing, and hardware-software co-design.



Daniel Báscones received the bachelor's degree in mathematics and computer science and the M.Sc. degree in computer science from the Complutense University of Madrid, Madrid, Spain, in 2016 and 2018, respectively.

He was a Research Associate at the Department of Computer Architecture and Automatics, Complutense University of Madrid, in 2016, where he is currently an Associate Professor. In 2020, he completed the Ph.D. thesis in the field of his research. His research interests include hyperspectral image compression on field-programmable gate arrays, dealing with fast lossless algorithms that aid with data transmission and more complex lossy algorithms for long-term storage.



Carlos González received the M.S. and Ph.D. degrees in computer engineering from the Complutense University of Madrid, Madrid, Spain, in 2008 and 2011, respectively.

He is currently an Associate Professor with the Department of Computer Architecture and Automation, Complutense University of Madrid, where he is also a Research Member of the GHADIR Group, where he mainly focuses on applying runtime reconfiguration in aerospace applications. His research interests include remotely sensed hyperspectral imaging, signal and image processing, efficient implementation of large-scale scientific problems on reconfigurable hardware, and the acceleration of artificial intelligence algorithms applied to games.

Dr. Gonzalez attended the Design Competition of the 2009 and 2010 IEEE International Conference on Field-Programmable Technology and received the second prize in 2012. He received the Best Paper Award of an Engineer under 35 years old at the 2011 International Conference on Space Technology.



Daniel Mozos (Member, IEEE) received the B.S. degree in physics and the Ph.D. degree in computer science from the Complutense University of Madrid, Madrid, Spain, in 1986 and 1992, respectively.

He was the Dean of the Computer Science Faculty, Complutense University of Madrid, from 2010 to 2018, where he is currently a Full Professor with the Department of Computer Architecture and Automation and Leads the GHADIR Research Group on dynamically reconfigurable architectures. His research interests include design automation, computer architecture, and reconfigurable computing.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 05:56:59 UTC from IEEE Xplore. Restrictions apply.